

Estudo Comparativo de Recomendadores com Foco no Modelo SASRec

1st Hygo Freire Tagarro

Departamento de Informática

Universidade Federal do Espírito Santo (UFES)

Avenida Fernando Ferrari, 514 – Vitória – ES – Brazil

hygo.tagarro@edu.ufes.br

Abstract—Recommender systems have become increasingly important to help users navigate the vast amount of digital content available today. While traditional methods like collaborative filtering and hybrid approaches still perform well, they often fail to consider the sequential nature of user interactions. In this work, we implement and evaluate a streamlined version of the SASRec model, which applies self-attention to capture temporal patterns in user behavior. The code was restructured for better modularity and tested on two widely used datasets: MovieLens-1M and MovieLens-20M. We also compare the model against popular baselines, such as SAR, ALS, NCF, BiVAE, LightGCN, and FastAI-based models. Our results show that SASRec consistently delivers better ranking quality, especially in larger and more complex datasets.

Index Terms—Recommender Systems, Sequential Recommendation, SASRec, Self-Attention, Deep Learning, MovieLens, nDCG, Collaborative Filtering

I. INTRODUÇÃO

Com o crescimento da internet e das plataformas online, sistemas de recomendação se tornaram peças chave para auxiliar usuários a encontrar conteúdos relevantes. Eles já fazem parte do dia a dia em diversas plataformas, como e-commerces, serviços de streaming, redes sociais, portais de notícias e outros ambientes online, com o objetivo de tornar a experiência única para o usuário aumentando assim o engajamento com o produto final.

Abordagens tradicionais, como filtro colaborativo [1], filtragem baseada em conteúdo [2] e abordagens híbridas [3], são muito utilizadas atualmente, mas encontram algumas limitações, tais como em cenários com uma quantidade limitada de dados ou o contexto as interações não é considerado de forma adequada. Um dos objetivos desses sistemas é sugerir aos usuários itens que sejam relevantes ou de interesse, com base no comportamento anterior de outros usuários. Entretanto, esses modelos levam em conta as preferências gerias dos usuários sem considerar a ordem e o tempo de cada interação, quando essas características são importantes entramos no campo da recomendação sequencial.

O desafio da recomendação sequencial é capturar padrões que aparecem somente ao longo das ações dos usuários. Isso exige modelos tenham a capacidade de entender dinâmicas temporais e contextuais — ou seja, o histórico de ações é importante pois por meio dele é possível dar mais importância para aquelas itens que são mais relevantes no momento.

Modelos como Cadeias de Markov (MCs) e Redes Neurais Recorrentes (RNNs) já foram bastante utilizados para isso, mas cada um tem suas limitações: MCs funcionam bem em cenários com dados esparsos, mas simplificam demais a dinâmica de transições; enquanto RNNs, apesar de mais expressivos, demandam muitos dados para treinar bem [5].

Este trabalho explora uma implementação do modelo SASRec (Self-Attentive Sequential Recommendation), que utiliza mecanismos de atenção para capturar dependências entre interações passadas de um usuário. A proposta original do SASRec já demonstrou bons resultados em tarefas de recomendação sequencial, especialmente por sua capacidade de focar em itens relevantes ao longo do histórico de ações do usuário.

A contribuição deste trabalho é implementar uma versão com melhores práticas de escrita de código mantendo a funcionalidade original e simplificando o processo de testes no futuro, com melhorias estruturais no código, além de aplicar o modelo a um conjunto de dados maior **MovieLens-20M** do que o utilizado originalmente. Com isso, buscamos avaliar o desempenho do modelo em cenários práticos, aproximando a análise de situações do mundo real.

Além disso, realizamos uma comparação sistemática entre o SASRec e outros modelos de recomendação amplamente utilizados, incluindo o SAR, ALS, NCF, BiVAE, LightGCN e abordagens baseadas no framework FastAI. Essa comparação visa entender as vantagens e limitações do SASRec frente a métodos tradicionais e modernos, avaliando aspectos como precisão, robustez e escalabilidade.

II. TRABALHOS CORRELATOS

O modelo SASRec (Self-Attentive Sequential Recommendation) [5] foi um dos primeiros a aplicar o mecanismo de self-attention para recomendação de itens de forma sequencial, por meio de Transformers, dando a possibilidade do modelo aprender padrões complexos nas interações dos usuários que muitas vezes não seriam utilizados nos modelos tradicionais.

Desde então, diversos trabalhos propuseram variações e extensões do SASRec. O BERT4Rec [6] adapta o modelo BERT para reconstruir itens mascarados em sequências, permitindo capturar dependências bidirecionais. Já o S3-Rec [7] introduz tarefas auto-supervisionadas para reforçar o aprendizado mesmo com poucos dados explícitos.

Além das variações conceituais, surgiram implementações que buscam escalar ou melhorar o SASRec em diferentes contextos. O projeto Scalable-SASRec [8] propõe otimizações para aplicar o modelo em grandes volumes de dados. O Hyperbolic-SASRec [9] investiga o uso de geometria hiperbólica para representar relações complexas entre itens.

Essas variantes mostram como o SASRec se tornou uma base sólida para pesquisas recentes, sendo continuamente adaptado para diferentes objetivos, arquiteturas e tipos de dados.

III. METODOLOGIA

Nesta seção, apresento como o modelo foi implementado, testado e avaliado ao longo do projeto. Esse modelo é baseado no modelo **SASRec** com modificações estruturais no código e testes com datasets maiores.

A. Arquitetura Base: SASRec

O **SASRec (Self-Attentive Sequential Recommendation)** é um modelo de recomendação sequencial baseado em atenção. Ele utiliza o mecanismo de *self-attention* para aprender quais interações anteriores de um usuário são mais relevantes para prever a próxima.

A entrada para o modelo é uma sequência de interações de usuários, na forma de embeddings de itens. Esses embeddings são passados por camadas de atenção que conseguem aprender padrão temporais e preferência dos usuários, após isso, é passado por uma camada que calcula a possibilidade de cada item ser o próximo interagido.

B. Pré-processamento dos Dados

O modelo recebe como entrada uma sequência ordenada de interações por usuário. Antes de treinar, os dados passam por um processo de pré-processamento que inclui:

- Filtragem de usuários e itens com poucas interações;
- Remoção de duplicatas;
- Conversão das sequências em listas indexadas por ID;
- Separação dos dados em treino, validação e teste, utilizando o método *leave-one-out*.

Esse pipeline segue as boas práticas propostas na literatura e é compatível com a implementação original do SASRec.

C. Treinamento

O treinamento é feito com otimização do erro de previsão, utilizando a função de perda de entropia cruzada (*Cross-Entropy Loss*) com amostragem negativa. Ou seja, para cada item positivo (realmente interagido pelo usuário), o modelo também vê alguns itens negativos (aleatórios), para aprender a diferenciá-los.

D. Comparação com Outros Modelos

Para avaliar o desempenho do modelo, ele foi comparado com os seguintes métodos:

- **SAR**: modelo baseado em regras, usado como baseline simples;

TABLE I
HIPERPARÂMETROS UTILIZADOS NO TREINAMENTO

Parâmetro	Valor
Tamanho do embedding	64
Número de camadas	2
Número de cabeças	2
Dropout	0.2
Batch size	128
Otimizador	Adam
Taxa de aprendizado	0.001

- **ALS**: fatoração de matrizes por mínimos quadrados alternados;
- **NCF**: filtragem colaborativa neural;
- **BiVAE**: autoencoder variacional bilateral;
- **LightGCN**: recomendação baseada em grafos;
- **FastAI**: modelo baseado em embedding e camadas lineares;

Todos os modelos foram avaliados sob as mesmas condições experimentais, utilizando os mesmos dados, métricas e divisão de treino/teste, conforme descrito na próxima seção.

E. Metodologia de Avaliação

Para avaliar o desempenho dos modelos de recomendação, foram utilizadas duas métricas amplamente adotadas na literatura de recomendação sequencial: **Hit Ratio** (HR@K) e **Normalized Discounted Cumulative Gain** (nDCG@K).

F. Hit Ratio (HR@K)

A métrica **HR@K** mede se o item relevante (ou de teste) está entre os K itens recomendados pelo modelo. Ela retorna 1 se o item correto estiver presente na lista e 0 caso contrário. A média dos resultados sobre todos os usuários fornece o valor final da métrica:

$$HR@K = \frac{1}{|U|} \sum_{u \in U} \mathbb{I}(\text{item relevante} \in \text{Top-}K_u) \quad (1)$$

G. Normalized Discounted Cumulative Gain (nDCG@K)

A métrica **nDCG@K** avalia a posição do item relevante na lista de recomendações. Quanto mais próximo do topo da lista o item relevante estiver, maior será a pontuação. A ideia é penalizar modelos que recomendam o item correto, mas apenas nas últimas posições.

$$DCG@K = \sum_{i=1}^K \frac{rel_i}{\log_2(i+1)} \quad (2)$$

$$IDCG@K = \sum_{i=1}^K \frac{rel_i^*}{\log_2(i+1)} \quad (3)$$

$$nDCG@K = \frac{DCG@K}{IDCG@K} \quad (4)$$

Em que rel_i representa a relevância binária do item na posição i , e $IDCG@K$ é o DCG ideal, usado para normalizar o valor.

IV. EXPERIMENTOS

Todos os experimentos foram realizados no Google Colab, utilizando uma GPU NVIDIA L4, executando o sistema operacional Ubuntu 22.04.4 LTS. O ambiente contava com 53 GB de memória RAM e uma GPU com 24 GB de VRAM GDDR6, equipada com 7.424 núcleos CUDA e 232 núcleos Tensor (4ª geração), otimizados para tarefas de aprendizado profundo. Além disso, o processamento foi complementado por uma CPU Intel(R) Xeon(R) @ 2.20GHz, com 12 núcleos físicos.

O conjunto de dados utilizado nos experimentos foi o **MovieLens-1M** e **MovieLens-20M**, um benchmark tradicional para avaliação de sistemas de recomendação. Para garantir consistência nos resultados, usuários e itens que apareciam apenas no conjunto de teste foram removidos, e a separação dos dados foi feita utilizando o método *leave-one-out*, onde a última interação de cada usuário é usada como teste.

V. RESULTADOS

Nesta seção, comparamos os resultados obtidos pelo modelo SASRec implementado neste trabalho com outros modelos clássicos e modernos da literatura. A Tabela II apresenta os valores das principais métricas utilizadas, com foco na métrica **nDCG@K**, que leva em conta a posição dos itens relevantes nas recomendações.

Todos os modelos foram avaliados no mesmo conjunto de dados (**MovieLens-1M**) e sob as mesmas condições de pré-processamento e divisão de treino/teste. Isso garante uma comparação justa e consistente entre os diferentes algoritmos. Os resultados dos modelos *BiVAE*, *LightGCN*, *SAR*, *ALS*, *NCF* e *FastAI* foram obtidos a partir de experimentos realizados no repositório complementar deste trabalho [10].

TABLE II
DESEMPENHO DOS ALGORITMOS COM BASE EM NDCG@K
(MOVIELENS-1M)

Algoritmo	nDCG@K
sasrec (este trabalho)	0.442171
bivae	0.410565
lightgcn	0.359210
sar	0.345610
fastai	0.190846
ncf	0.154868
als	0.096373

Como mostrado na Tabela II, o modelo **SASRec** alcançou o melhor desempenho em termos de **nDCG@K**, superando todos os outros algoritmos avaliados. Isso confirma a capacidade do SASRec de capturar melhor a ordem das interações dos usuários, principalmente em tarefas sequenciais, o que é coerente com os resultados da literatura [5].

A. Resultados no MovieLens-20M

Além do conjunto MovieLens-1M, também avaliamos o desempenho do modelo SASRec no conjunto de dados **MovieLens-20M**, que é mais robusto e desafiador por conter um número significativamente maior de interações. O objetivo

foi analisar como o modelo se comporta em um cenário mais realista, com maior densidade de dados.

Durante o treinamento, observamos uma evolução consistente da métrica **nDCG@K** tanto para os dados de treino quanto para validação. As Figuras 1 e 2 mostram a curva de aprendizado ao longo de 50 épocas.

Os gráficos indicam que o modelo apresenta uma convergência estável a partir da 25ª época, com **nDCG@K** atingindo valores superiores a 0.92 tanto no treino quanto na validação. Esse resultado reforça a capacidade do SASRec de generalizar bem em grandes volumes de dados, mantendo alta precisão mesmo em cenários mais complexos.

REFERENCES

- [1] J. L. Herlocker, J. A. Konstan, A. Borchers, and J. Riedl, "An algorithmic framework for performing collaborative filtering," in *Proc. 22nd Annual Int. ACM SIGIR Conf. Research and Development in Information Retrieval (SIGIR '99)*, Berkeley, CA, USA, 1999, pp. 230–237, doi: 10.1145/312624.312682.
- [2] Y. Xia, G. Di Fabbrizio, S. Vaibhav, and A. Datta, "A content-based recommender system for e-commerce offers and coupons," in *Proc. ACM SIGIR Workshop on eCommerce (eCOM@SIGIR)*, 2017. [Online]. Available: <https://api.semanticscholar.org/CorpusID:59528386>
- [3] E. Çano and M. Morisio, "Hybrid recommender systems: A systematic literature review," *Intelligent Data Analysis*, vol. 21, no. 6, pp. 1487–1524, Nov. 2017, doi: 10.3233/IDA-163209. [Online]. Available: <http://dx.doi.org/10.3233/IDA-163209>
- [4] W.-C. Kang and J. McAuley, "Self-attentive sequential recommendation," in *Proc. 2018 IEEE Int. Conf. on Data Mining (ICDM)*, Singapore, 2018, pp. 197–206, doi: 10.1109/ICDM.2018.00035.
- [5] W.-C. Kang and J. McAuley, "Self-Attentive Sequential Recommendation," in *Proc. 2018 IEEE Int. Conf. on Data Mining (ICDM)*, Singapore, 2018, pp. 197–206, doi: 10.1109/ICDM.2018.00035.
- [6] F. Sun, J. Liu, J. Wu, C. Pei, X. Lin, W. Ou, and P. Jiang, "BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer," *arXiv preprint arXiv:1904.06690*, 2019. [Online]. Available: <https://arxiv.org/abs/1904.06690>
- [7] K. Zhou et al., "S3-Rec: Self-Supervised Learning for Sequential Recommendation with Mutual Information Maximization," in *Proc. 29th ACM Int. Conf. on Information and Knowledge Management (CIKM)*, Oct. 2020, pp. 1893–1902, doi: 10.1145/3340531.3411954.
- [8] G. Mezentsev, D. Gusak, I. Oseledets, and E. Frolov, "Scalable Cross-Entropy Loss for Sequential Recommendations with Large Item Catalogs," in *Proc. 18th ACM Conf. on Recommender Systems (RecSys '24)*, 2024, pp. 475–485, doi: 10.1145/3640457.3688140.
- [9] E. Frolov, T. Matveeva, L. Mirvakhabova, and I. Oseledets, "Self-Attentive Sequential Recommendations with Hyperbolic Representations," in *Proc. 18th ACM Conf. on Recommender Systems (RecSys '24)*, Bari, Italy, 2024, pp. 981–986, doi: 10.1145/3640457.3688180.
- [10] H. F. Tagarro, "CDD - Projeto Final: Avaliação de Algoritmos de Recomendação," GitHub, 2025. [Online]. Available: <https://github.com/hygo2025/cdd-projeto-final> [Accessed: Mar. 27, 2025].

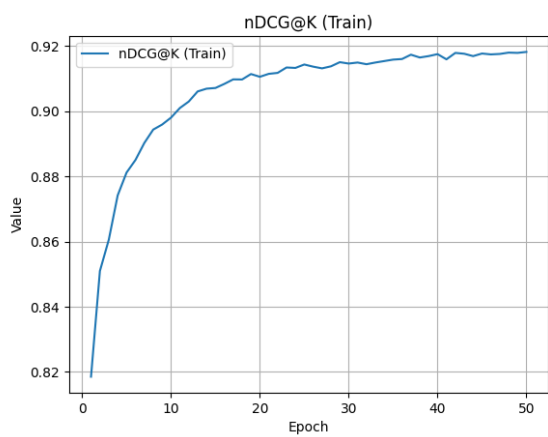


Fig. 1. Evolução do nDCG@K durante o treinamento (MovieLens-20M)

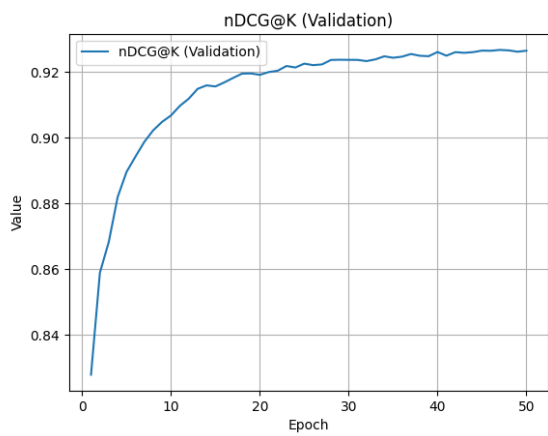


Fig. 2. Evolução do nDCG@K durante a validação (MovieLens-20M)